Day 1 Exercise - From Rules to Generated Images

Day 1 wrap-up (self-paced, do it any time)

One short exercise that revisits everything from Day 1 in a single Colab notebook. You build it step by step. If you get stuck, ask your AI assistant (ChatGPT, Gemini, Claude, or the assistant in your editor) to write the code for each step, paste it into Colab, and run it.

You do not need to write code from scratch. Each part below has a ready-to-use prompt you can hand to an LLM.

What you will practice

1. **Session 1:** train and evaluate a classifier, read accuracy and the other metrics, spot overfitting.
2. **Session 2:** build and train a small neural network in PyTorch.
3. **Session 3:** train a tiny VAE and generate brand-new samples.
4. **Session 4:** generate an image from a pre-trained diffusion pipeline (optional).

Setup

1. Open Google Colab and create a new notebook.
2. Keep the free CPU runtime for Parts 1 to 3. For Part 4, switch to the free T4 GPU runtime (Runtime, then Change runtime type).
3. Install anything a step needs with `!pip install ...` in the first cell.

How to use an LLM for this

For each part, copy the prompt in the code block into your AI assistant, then copy its answer into a Colab cell and run it. If a cell errors, paste the error message back to the assistant and ask it to fix the code.

The exercise

Part 1: Train and evaluate a classifier

Load a small dataset, split it, train a model, and check how good it is.

```
Write Python for Google Colab that:
1. Loads the scikit-learn digits dataset (load_digits).
2. Splits it into train and test sets (test_size=0.25, random_state=42).
3. Trains a LogisticRegression classifier.
4. Prints accuracy, precision, recall, and F1 on the test set.
5. Prints the training accuracy too, so I can compare train vs test.
Keep it short and add one comment per step.
```

You should see: test accuracy around 0.95 or higher, and training accuracy close to the test accuracy (a small gap means little overfitting).

Part 2: A small neural network in PyTorch

Do the same classification task with a neural network and watch the loss drop.

```
Write PyTorch code for Google Colab that:
1. Loads the scikit-learn digits dataset and scales the pixels to 0-1.
2. Splits into train and test tensors.
3. Builds a small feedforward network (input 64, one hidden layer of 32
with ReLU, output 10).
4. Trains for 50 epochs with cross-entropy loss and the Adam optimizer.
5. Prints the loss every 10 epochs and the final test accuracy.
Keep it under 40 lines and add short comments.
```

You should see: the loss going down each time it prints, and a final test accuracy in the same range as Part 1.

Part 3: Generate new digits with a VAE

Train a tiny variational autoencoder, then sample it to create new images.

```
Write a minimal variational autoencoder in PyTorch for Google Colab that:
1. Uses the scikit-learn digits dataset (8x8 images, flattened to 64).
2. Has an encoder to a 2-dimensional latent space and a decoder back to 64.
3. Uses the reparameterization trick and a loss of reconstruction + KL
divergence.
4. Trains for a few epochs on CPU (keep it fast).
5. After training, samples 8 random points from the latent space and shows
the decoded images with matplotlib.
Explain each block briefly in comments.
```

You should see: blurry but digit-like images that are not copies of the training data. That is generation: sampling from a learned distribution.

Part 4: Generate an image with diffusion (optional)

Use a ready-made model, no training. Switch Colab to the T4 GPU runtime first.

```
Write Colab code that loads a small pre-trained diffusion pipeline from the
Hugging Face diffusers library and generates one image. Use a small number
of
inference steps so it finishes quickly, set a seed for a repeatable result,
and
display the image. List any pip installs I need.
```

You should see: one generated image. Try changing the number of steps or the seed and run it again.

Success checklist

- [ ] Part 1 prints all four metrics and a train-vs-test comparison
- [ ] Part 2 shows the loss dropping and a final accuracy
- [ ] Part 3 displays newly generated digit images
- [ ] Part 4 displays one diffusion-generated image (optional)

Stretch goals

- In Part 1, try a second model (like a decision tree with a large depth) and compare its train and test accuracy to see overfitting clearly.
- In Part 2, add a second hidden layer and see if accuracy changes.
- In Part 3, pick two latent points and decode the points along the line between them to watch one digit morph into another.

Getting stuck?

Each part maps to a full worked lab from today. Open the matching `solution/` notebook for reference:

Part	Reference lab
Part 1	Lab 1.1
Part 2	Lab 1.2
Part 3	Lab 1.3
Part 4	Lab 1.4